Scheduling and Offloading Tasks from Connected Vehicles to Roadside Edge Servers

Project report

Gaurav, Anukul, Shivam, Dhruvitaraje

Abstract

Modern cars run applications (for example, detecting obstacles) that are made of several small steps, some of which must wait for others. A car can send these steps to nearby roadside computers, called *edge servers*, to get them done faster. This project builds a simulator, driven by realistic traffic from the SUMO traffic simulator, and compares different ways of deciding *which server* should run each step and *in what order* the steps should be sent. Our main method, **EFT (Earliest Finish Time)**, gave the fastest results in every test, beating two simple rules and a machine-learning (ML) model, on both a synthetic workload and a workload built from a public real-world dataset.

1 The problem in plain words

Imagine a car that needs to run a small application, such as *"read the camera, find pedestrians, find lanes, then decide how to steer"*. This application is split into **tasks**. Some tasks depend on others: you cannot decide how to steer before the pedestrians have been found. Drawn as boxes and arrows, this is called a **DAG** (Directed Acyclic Graph): arrows show "must finish before", and there are no loops.

The car does not run these tasks itself. It sends them over a wireless link to one of four **edge servers** placed beside the road. Two questions have to be answered over and over:

1. **Offloading:** which server should run this task? A nearby server gives a fast upload, but it may already be busy. A faraway server may be idle and powerful, but the upload takes longer.
2. **Scheduling:** when several tasks are waiting, which one should be sent first?

What makes this harder than ordinary scheduling is that **cars move**. The distance between a car and each server, and therefore the upload time, changes every fraction of a second.

What we measure: the **makespan** of each DAG, the time from when the car asks for the work until the last task of that DAG is finished. Smaller is better.

2 What we built

2.1 The simulator

We wrote a simulator in C++ that repeats the following steps every 0.1 seconds of simulated time:

1. Ask the **SUMO** traffic simulator where every car is now. SUMO is a widely used, open-source tool that simulates real road traffic (traffic lights, turning, queues of cars).

2. Let each server work on its tasks; finished tasks "unlock" the tasks that were waiting for them.
3. Randomly create new DAGs, each belonging to a randomly chosen car.
4. Decide the order of waiting tasks (scheduling) and a server for each one (offloading).
5. Record measurements.

Every task goes through the same life cycle: *ready* → *wait for the next decision* → *upload* → *queue at the server* → *execute* → *done*. Upload time grows with distance; execution time depends on how fast the server is.

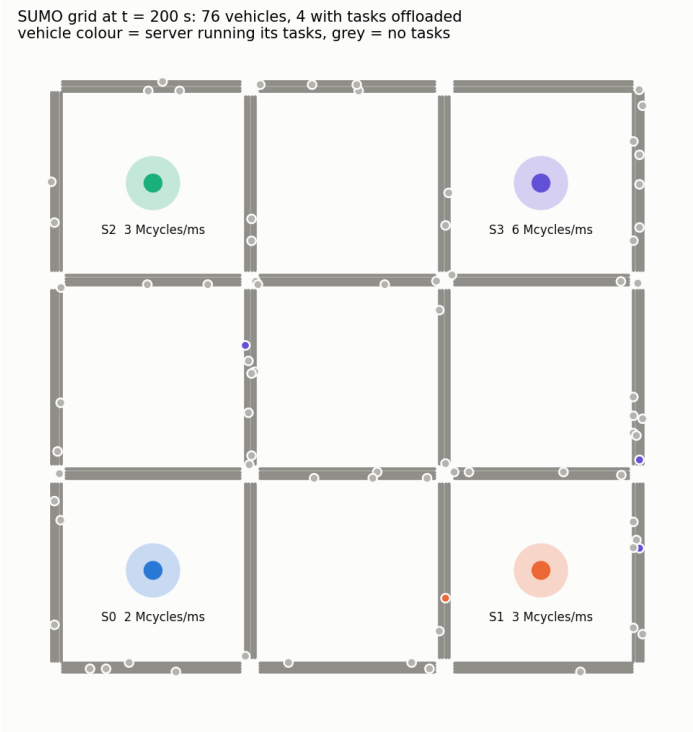


Figure 1: The simulated city: a 4×4 grid of streets (600 m × 600 m). The four coloured circles are the edge servers; S3 is the fastest. Each dot is a car from SUMO. A coloured car currently has tasks running on the server of that colour; grey cars have none.

2.2 The setup

Item	Value
Road network	SUMO 4×4 grid, 200 m blocks
Traffic	500 cars over 500 s; 56–86 cars on the road at once
Edge servers	4 servers with speeds 2, 3, 3 and 6 (S3 is three times faster than S0)
Workload 1 (synthetic)	random DAGs with 3–6 tasks
Workload 2 (real)	9 real workflow DAGs (22–104 tasks) from the public WfCommons dataset
Load	20%, 40%, 60%, 80% of the total server capacity
Repetitions	10 different random seeds per setting; 560 runs in total

Table 1: Experimental setup.

2.3 The public dataset: WfCommons WfInstances

Random DAGs are useful, but they are not how real applications look. To test our methods on *real* task graphs, we used the public **WfInstances** dataset from the **WfCommons** project (`https://github.com/wfcommons/WfInstances`, licence LGPL-3.0; Coleman et al., *Future Generation Computer Systems*, 2022). WfCommons is widely used in workflow and DAG-scheduling research.

What is in it. Each file is a recording of one real run of a scientific computing pipeline on a cluster. It lists every task, which tasks it depends on, how long it actually ran, and the size of every file it read and wrote. In other words, it is a real DAG with real task sizes.

What we took from it. We picked the smallest recorded run of each of 9 different applications, so that the DAG shapes are as varied as possible:

Workflow	What it does	Tasks	Shape
1000Genome	human genome analysis	52	wide, 3 levels
BLAST	DNA sequence search	43	split, many in parallel, merge
BWA	DNA sequence alignment	104	wide, 3 levels
Cycles	crop and soil simulation	67	4 levels
Epigenomics	genome methylation analysis	41	parallel chains, 9 levels
Montage	stitching telescope images	58	8 levels, very uneven task sizes
Seismology	earthquake signal processing	101	one wide level, then merge
SoyKB	soybean genome analysis	96	deep, 11 levels
SRASearch	searching DNA archives	22	split, parallel, merge
Total		**584**	

Table 2: The 9 real workflows used from WfCommons.

How we used it. A small script (`data/prepare_wfcommons.py`) does four things:

1. **Downloads** the 9 files from a fixed version of the dataset (a pinned commit), so everyone gets exactly the same data.
2. **Keeps the real structure:** every dependency ("task B needs task A") is kept exactly as recorded.
3. **Keeps the real relative sizes:** a task's amount of work comes from its recorded running time, and its upload size comes from the size of its recorded input files.
4. **Rescales to vehicle scale:** the original tasks ran for seconds to hours on big cluster machines, which is far too large for a car. So each workflow is scaled so that its *average* task has the same work (300 million CPU cycles) and data (225 KB) as our synthetic tasks. A task that was 10 times bigger than average in the original is still 10 times bigger after scaling.

During a simulation, every time a car starts a new application, one of the 9 workflows is picked at random and becomes that car's DAG.

In short: the dataset gives us *realistic DAG shapes and realistic differences in task size.* It is not a recording of real car applications (we did not find a public dataset of real vehicle-application DAGs), so we use it to check that our conclusions still hold on real, irregular task graphs, not only on small random ones.

2.4 The methods we compare

Offloading (which server):

- **EFT – Earliest Finish Time (our method).** For every server, estimate when the task would finish:

$$\text{finish} = \max(\text{upload time},\ \text{time until the server is free}) + \frac{\text{work}}{\text{server speed}}$$

  and pick the server with the smallest value. It considers distance, how busy the server is, and how fast it is, all at once.

- **ML (machine learning).** A small neural network learns to predict the finish time from examples, and the task goes to the server with the lowest prediction (details in Section 3.4).
- **Least-loaded.** Pick the least busy server; ignore distance.
- **Nearest.** Pick the closest server; ignore how busy it is.

Scheduling (which waiting task first):

- **HEFT-lite.** Send first the task that has the longest chain of work still waiting behind it in its DAG (the "critical path"). This is the ordering idea of the well-known HEFT algorithm.
- **FIFO.** First come, first served.

3 Results

3.1 Which server? EFT is the fastest

Load	EFT (ours)	ML	Least-loaded	Nearest
Synthetic DAGs (3–6 tasks)				
20%	**0.79**	0.79	1.05	0.97
40%	**0.83**	0.84	1.02	1.38
60%	**0.91**	0.92	1.07	3.71
80%	**1.14**	1.16	1.28	12.24
Real workflow DAGs (22–104 tasks)				
20%	**3.24**	3.49	4.05	10.25
40%	**3.89**	4.15	4.69	15.18
60%	**5.42**	5.78	6.14	26.45
80%	**9.69**	10.08	10.24	41.95

Table 3: Average makespan in seconds (from `results/aggregate.csv`). EFT has the lowest value in every row.

How to read this: at 80% load on synthetic DAGs, a car waits on average 1.14 s with EFT, but 12.24 s with Nearest, more than ten times longer. Nearest fails because everyone near a busy road sends work to the same server, which overflows while other servers stay idle.

3.2 A fair head-to-head comparison

Each run has different traffic and different DAGs, so results jump around from run to run. To compare methods fairly, we ran every method on *exactly the same* traffic and DAGs (same seed), and measured how much slower each one was than EFT.

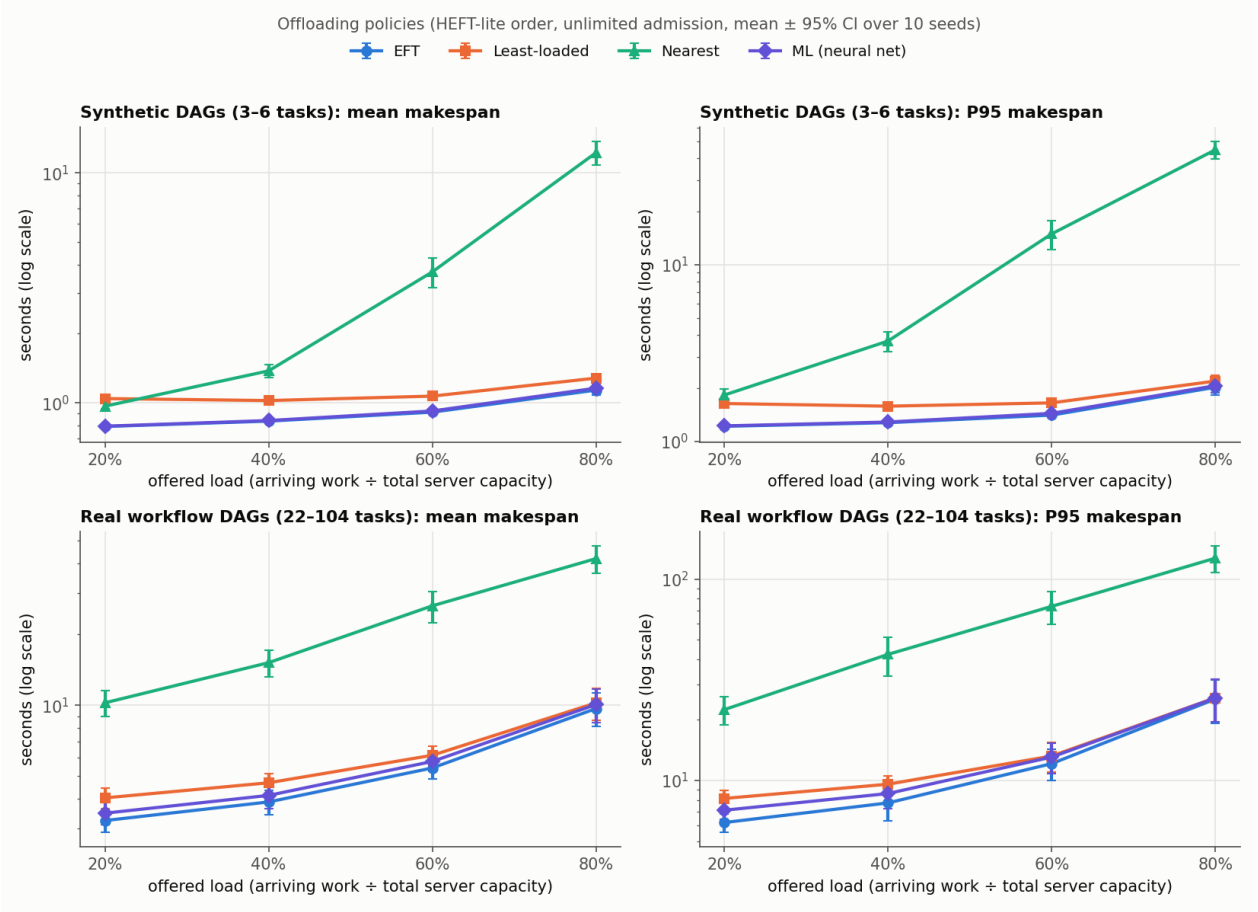


Figure 2: Makespan of the four offloading methods as the system gets busier (x-axis). Top: synthetic DAGs; bottom: real DAGs. Left: average; right: 95th percentile (the slow cases). Lower is better. The y-axis is logarithmic so that all methods fit on one chart. Error bars show 95% confidence intervals over 10 seeds.

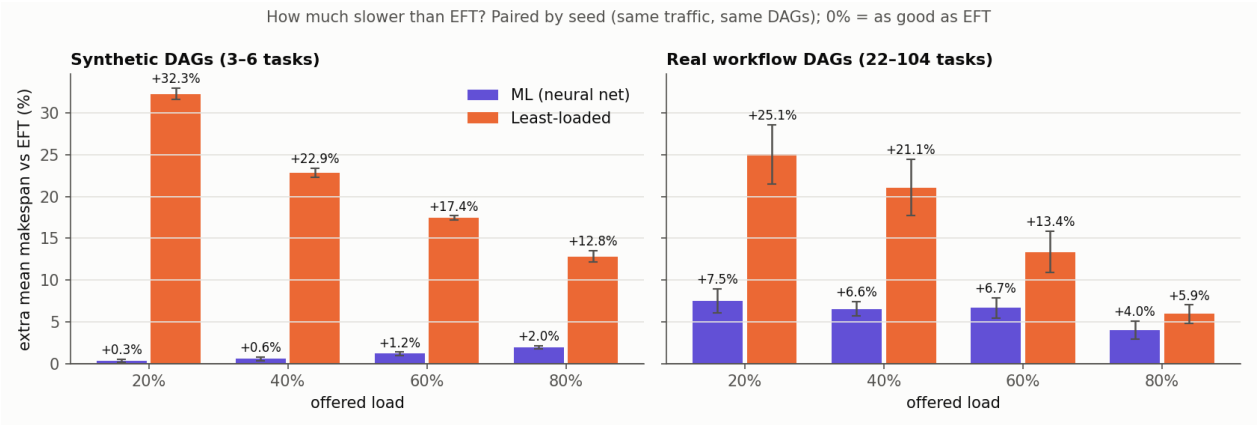


Figure 3: How much slower each method is than EFT, on identical traffic and DAGs. 0% would mean "as good as EFT". Every bar is above zero.

	ML vs EFT	**Least-loaded vs EFT**
Synthetic DAGs	0.3% to 2.0% slower	13% to 32% slower
Real workflow DAGs	4.0% to 7.5% slower	6% to 25% slower
EFT better in	79 of 80 runs	80 of 80 runs

Table 4: Paired comparison (from `results/paired_vs_eft.csv`). Nearest is left out because it is 23% to more than 10 times slower.

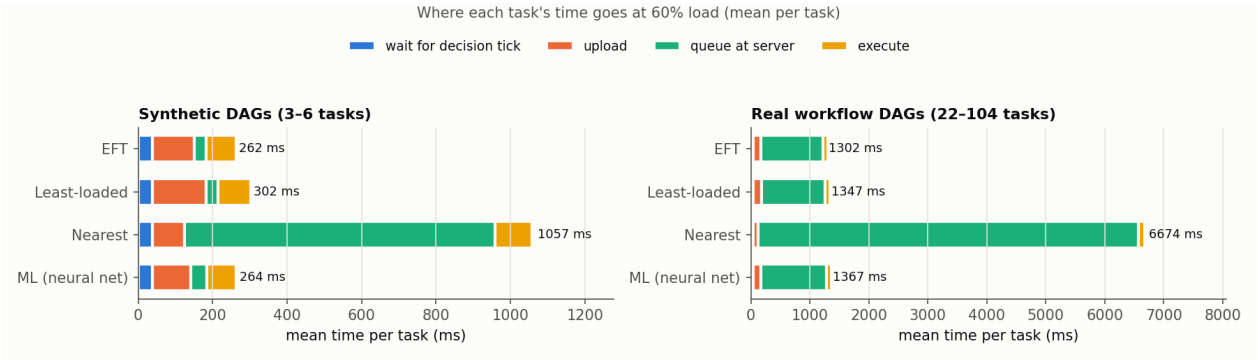


Figure 4: Where the time of an average task goes at 60% load: waiting for the next decision, uploading, queueing at the server, and executing.

3.3 Why do the simple rules lose?

Figure 4 splits each task's time into its parts:

- **Nearest** has the shortest uploads, but its tasks spend most of their time *queueing* (the long green bar), because it piles work onto the closest server.
- **Least-loaded** keeps queues short, but its uploads are longer because it happily picks servers far away.
- **EFT** balances both, because it adds them up before deciding.

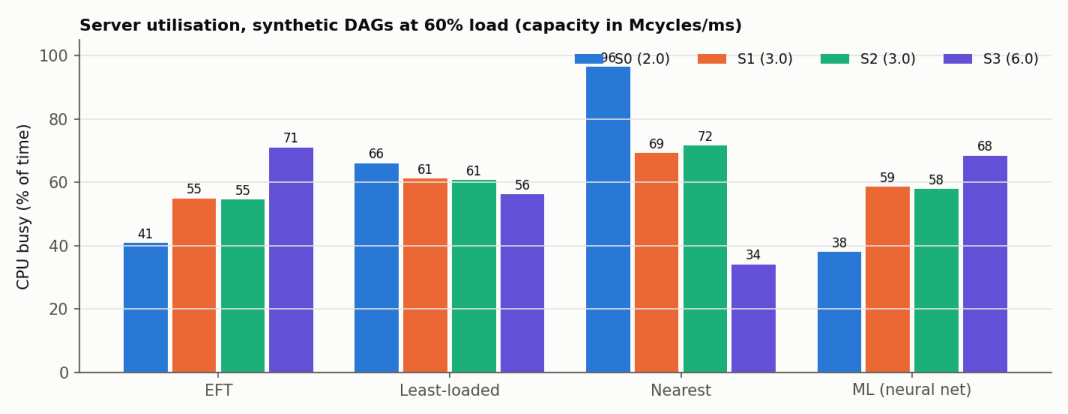


Figure 5: How busy each server is (synthetic DAGs, 60% load). EFT and ML give the most work to the fast server S3. Nearest overloads the slow server S0 (96% busy) while the fast server S3 is mostly idle (34%).

3.4 The machine-learning baseline

To check whether a learned model could do better than our formula, we trained a small **neural network** (2 layers of 32 neurons, using scikit-learn). It sees only six raw numbers for each task and server: distance, data size, amount of work, server speed, how much work is waiting at the server, and how many tasks are there. From these it predicts how long the task will take. It was trained on about 163 000 examples from separate simulation runs that were never used for testing.

Why EFT wins against ML: both make the same kind of decision ("pick the earliest finish"). EFT computes the finish time from a formula based on how uploads and queues work; the ML model has to *guess* it from examples. A more accurate estimate means better choices, so EFT is slightly faster.

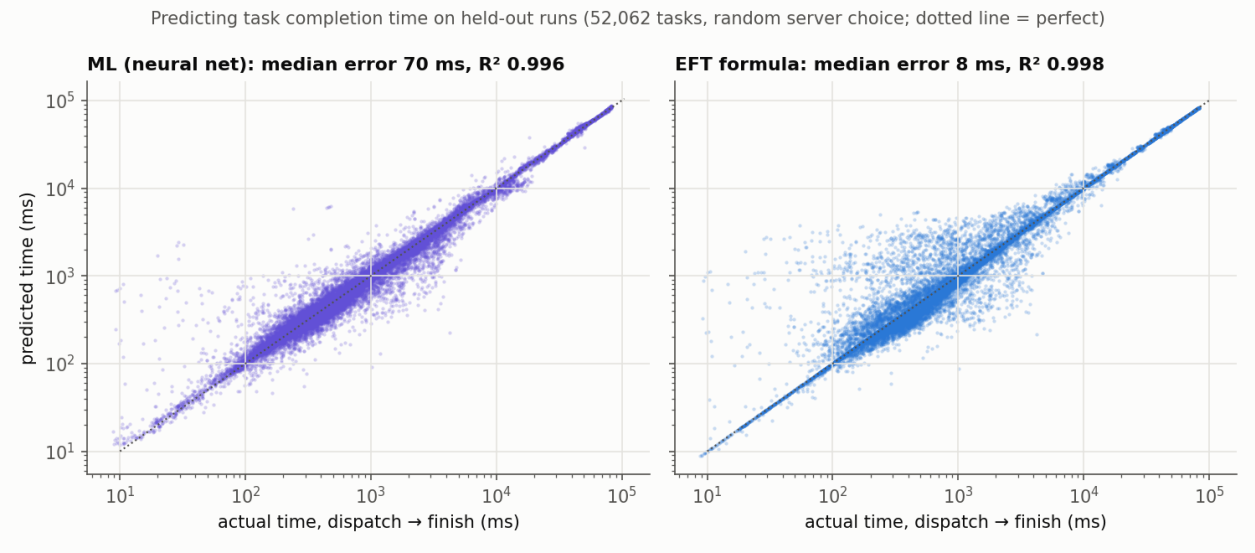


Figure 6: Predicted versus actual task time on unseen data. Points on the dotted line are perfect predictions. Both are good, but EFT's formula is tighter.

Predictor	Typical (median) error	Average error	R^2 (1 = perfect)
ML neural network	70 ms	308 ms	0.996
EFT formula	8 ms	221 ms	0.998

Table 5: Prediction accuracy on 52 062 unseen tasks (from `ml/training_report.json`).

3.5 In which order? HEFT-lite versus FIFO

When servers accept every task immediately, the order barely matters: HEFT-lite is only slightly faster (about 0.3–1%). To test the order properly, we also limited each server to 5 tasks at a time, so tasks must wait in line and the order decides who goes first.

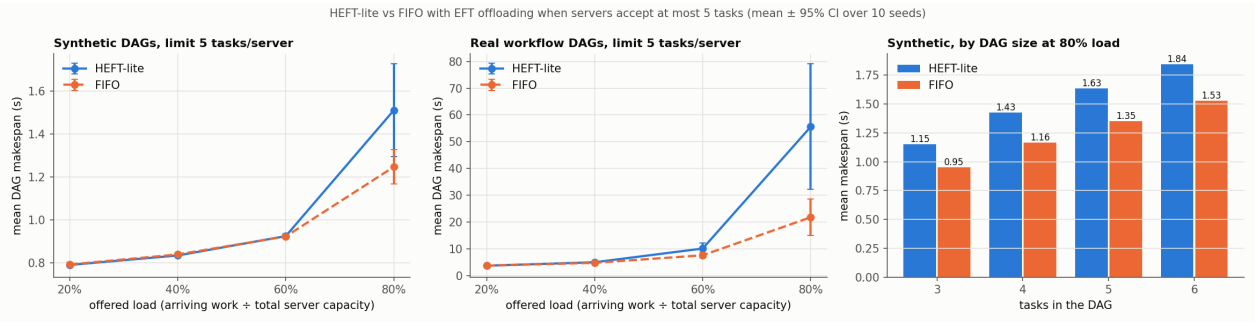


Figure 7: HEFT-lite versus FIFO when each server holds at most 5 tasks. Under heavy load HEFT-lite becomes much slower. Right: this happens for DAGs of every size.

Load	Synthetic DAGs	Real DAGs
20%	−0.3%	−0.7%
40%	−0.6%	+4.3%
60%	+0.2%	+32%
80%	+20%	+141%

Table 6: Change in makespan of HEFT-lite compared with FIFO, with the 5-task limit (negative = HEFT-lite is faster).

Why? HEFT gives the highest priority to the task with the most work still ahead of it, and that is always the *first* task of a brand-new DAG. So new DAGs keep jumping the queue, while old DAGs that are almost finished have to wait. HEFT was designed to order the tasks of *one* application, not many applications competing at the same time. This is a useful lesson: a famous algorithm is not automatically the right choice in a new setting.

3.6 One run over time

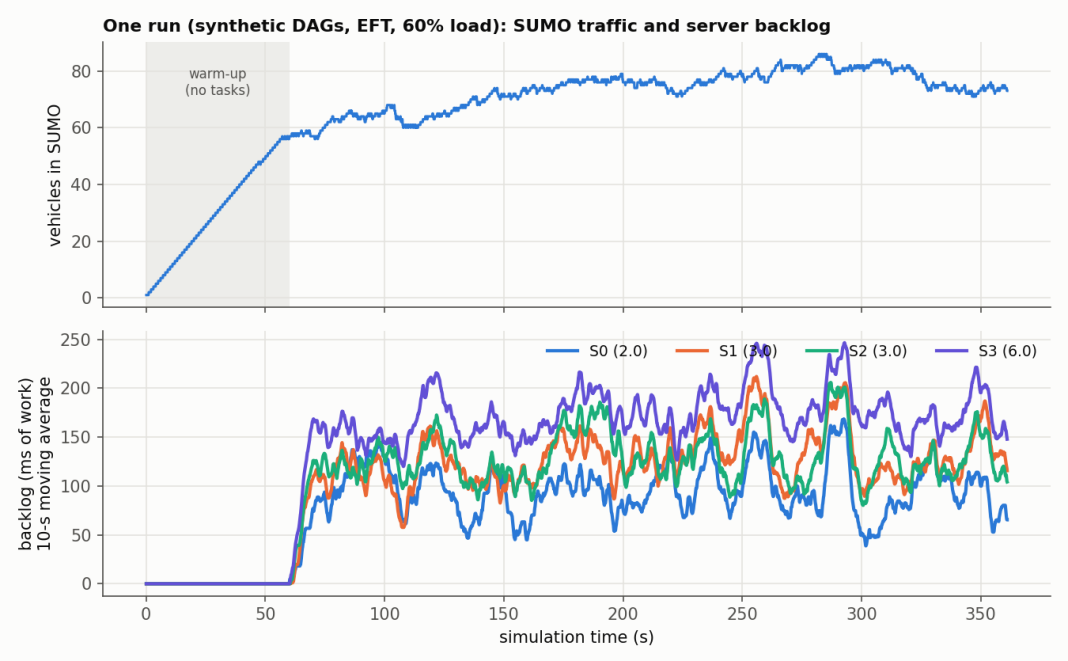


Figure 8: A single run with EFT. Top: number of cars in SUMO (the first 60 s is a warm-up with no tasks). Bottom: how much work is waiting at each server. Queues stay well under one second, and the fast server S3 takes the most work.

4 Why EFT is one of the best choices

Among all the methods we tested, EFT was the best in every setting. The reasons are simple:

1. **It looks at everything that matters.** Distance (upload time), how busy the server is (waiting time) and server speed (execution time) are all converted to the same unit, milliseconds, and added up. Each simple rule ignores at least one of them, and that is exactly where it loses time (Figure 4).
2. **It needs no training.** The ML model needed 32 extra simulation runs and about 163 000 examples, and was still less accurate. EFT works immediately and keeps working if servers or traffic change.
3. **It is fast to compute.** One short calculation per server, so deciding is practically instant even with many tasks.
4. **It is easy to explain.** Every decision can be checked by hand, which is not true for a neural network.
5. **It held up on real DAG shapes.** The advantage remained on the irregular real workflows from WfCommons, not just on random DAGs.

Being honest about the limits. Our simulator uses the same upload and queue rules that EFT's formula is based on, which gives EFT a home advantage over the ML model. On real roads, with real radio links that no simple formula describes exactly, a learned model might close the gap. Also, the link model is a simple distance formula (not a full wireless network simulator), the real dataset comes from scientific computing rather than cars, and we tested one city layout. So the fair conclusion is: *EFT is the best of the methods we tested and a strong, simple default choice*, and more realistic network models are the natural next step.

5 Summary

- We built a simulator for sending tasks from moving cars to roadside servers, using realistic traffic from SUMO, synthetic DAGs and real DAG shapes from the public WfCommons dataset.
- We compared four ways of choosing a server and two ways of ordering tasks, over 560 runs with 10 seeds each.
- **EFT was the fastest everywhere.** Compared with EFT, Least-loaded needed 6–32% more time, Nearest up to 11 times more, and a trained neural network 0.3–7.5% more.
- The famous HEFT ordering helps only slightly, and can hurt badly when many applications compete for limited servers.